AR-525: Reinforcement Learning for Robotics

Assignment 1: Grid Navigation using Dynamic Programming (UR5)

Submitted by:

Sushant (S25064)
Harsh Vardhan Saxena (S25068)

Instructor: Dr. Deepak Raina, CAIR, IIT Mandi

Date of Submission: February 07, 2026

1 Overview

In this assignment we solve a small grid navigation problem using Dynamic Programming (DP) and then visualize the resulting behavior on a UR5 setup in PyBullet. The goal is to find a policy that reliably moves from the start state to the goal while avoiding obstacle states that end the episode with a large penalty.

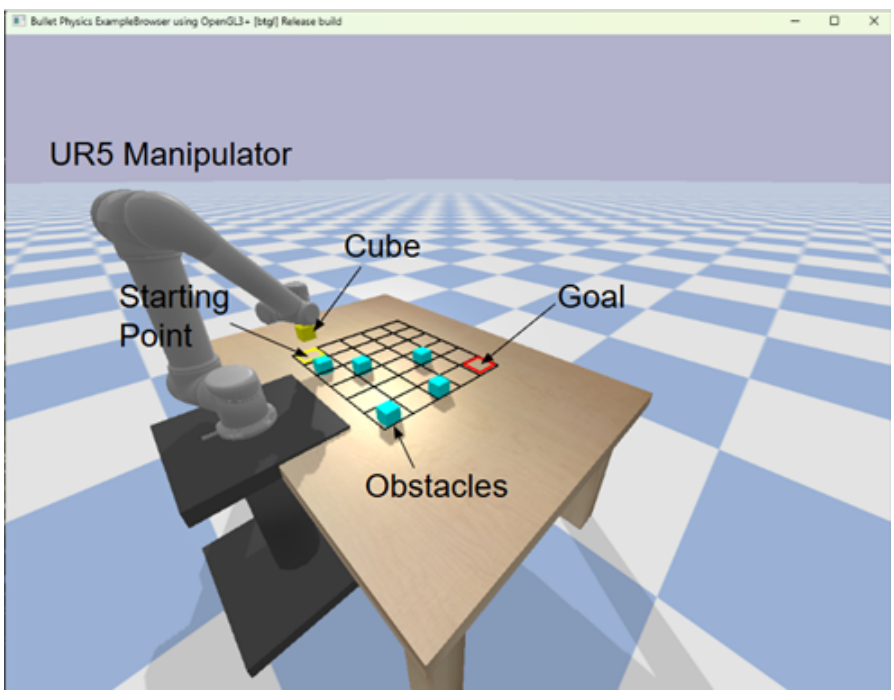

Figure 1: PyBullet environment used for the grid navigation task.

2 Problem Setup

2.1 Grid world

The environment is a deterministic Markov Decision Process (MDP) with:

- States: $S = \{0, 1, \ldots, mn - 1\}$ laid out on a `rows` $\times$ `cols` grid.
- Actions: $A = \{\text{LEFT}, \text{DOWN}, \text{RIGHT}, \text{UP}\}$ (4-connected moves).
- Transitions: deterministic; invalid moves (off-grid) keep the agent in the same state.
- Terminal states: reaching the goal terminates the episode; entering an obstacle state terminates the episode.

2.2 Rewards

From the implementation in `a1/utils.py` (transition model `env.P`):

- Step reward: -1 for non-terminal moves.
- Goal reward: $+10$ upon reaching the goal (terminal).
- Obstacle reward: -50 upon stepping into an obstacle (terminal).

2.3 Discount factor

Both algorithms are executed with $\gamma = 0.99$.

3 Implementation Details

3.1 Code organization

- `utils.py`: grid MDP and DP algorithms.
- `main.py`: experiment harness (random obstacles, timing, and PyBullet visualization).

3.2 MDP construction (`GridEnv`)

The class `GridEnv` defines:
(1) coordinate conversions (state $\leftrightarrow$ (row,col)),
(2) deterministic next-state function `_get_next_state`, and
(3) a transition dictionary `P[s][a]` of the form $P(s' \mid s, a) = 1$ with an associated reward and terminal flag.

Implementation note. In `GridEnv`, the obstacle list should be stored (e.g., `self.obstacles = obstacles`) before building the transition dictionary. Otherwise, references to `self.obstacles` will raise an error. In our setup, obstacles are treated as terminal states with a large negative reward.

3.3 Dynamic Programming algorithms

All DP algorithms are implemented in `a1/utils.py`.

3.3.1 Policy evaluation

Given a fixed policy $\pi(a \mid s)$, iterative policy evaluation updates

$$V(s) \leftarrow \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a)\big(R(s, a, s') + \gamma V(s')\big),$$

until the maximum change Δ is below θ.

3.3.2 Policy improvement

Given a value function V, compute

$$Q(s, a) = \sum_{s'} P(s' \mid s, a)\big(R(s, a, s') + \gamma V(s')\big),$$

then set a greedy deterministic policy $\pi(s) = \arg\max_a Q(s, a)$.

3.3.3 Policy iteration

Policy iteration alternates:

1. Evaluate the current policy to get V^π.
2. Improve greedily to get a new policy.

The algorithm terminates when the policy stops changing.

3.3.4 Value iteration

Value iteration repeatedly applies the Bellman optimality backup:

$$V(s) \leftarrow \max_a \sum_{s'} P(s' \mid s, a)\big(R(s, a, s') + \gamma V(s')\big)$$

until convergence, then extracts a greedy policy.

4 Experiment Procedure (`a1/main.py`)

4.1 Random obstacles

For each run, the script samples 16 obstacle states uniformly at random from the non-start and non-goal states (for an 8×8 grid).

4.2 Reward scenarios

The updated `main.py` evaluates the algorithms under two reward settings (same obstacles, same grid):

- **Dense:** step reward -1.
- **Sparse:** step reward 0.

In both cases, reaching the goal gives $+10$ and stepping into an obstacle gives -50 (terminal).

4.3 Metrics

The script reports:

- Execution time for policy iteration and value iteration (wall-clock).
- Convergence metadata (iterations / evaluation sweeps and final Bellman update deltas).
- Path metrics: path length, total reward, discounted return, and whether the goal is reached.
- Heatmaps of the learned value function (with the path overlaid).
- A bar-chart comparison across scenarios (runtime, path length, discounted return).

4.4 Visualization

The PyBullet visualization includes:

- The grid drawn on top of the table.
- Start/goal markers (colored squares).
- Obstacle blocks loaded at the sampled obstacle states.
- End-effector motion following the computed optimal path (stable IK), with a green trail.

4.5 Basic system positioning

For reproducibility, the main scene objects are placed as follows (as set in `a1/main.py`):

- **Grid size:** 8×8 with 16 obstacles.
- **Hyperparameters:** $\gamma = 0.99$ and $\theta = 10^{-6}$.
- **Camera:** `resetDebugVisualizerCamera(1.5, 45, -30, [0, -0.3, 0.65])`.
- **Table:** `table/table.urdf` at $[0, -0.3, 0]$ with `globalScaling=2.0`.
- **Robot stand:** `robot_stand.urdf` at $[0, -0.8, 0]$ (fixed base).
- **UR5 base:** `ur5.urdf` at $[0, -0.8, 0.65]$ (fixed base).
- **Grid mapping:** grid states are mapped to (x, y) positions around `table_center=[0, -0.3, 0.65]`$with$`grid_size=0.10`.
- **End-effector orientation:** a downward tool orientation is enforced with `ee_orn_euler=(0, -1.5`π, π).

Value heatmap. The value vector V is reshaped into a `rows` $\times$ `cols` grid and plotted using Seaborn. The selected optimal path is overlaid on the heatmap for easier interpretation.

5 Results and Comparison

Figures 2–4 show the saved heatmaps and the overall comparison plot from our runs.

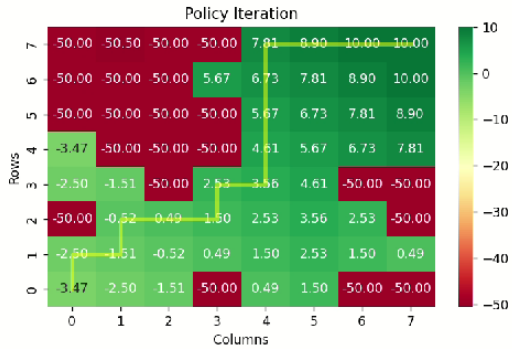

Figure 2: Value-function heatmap for **Policy Iteration** (saved output).

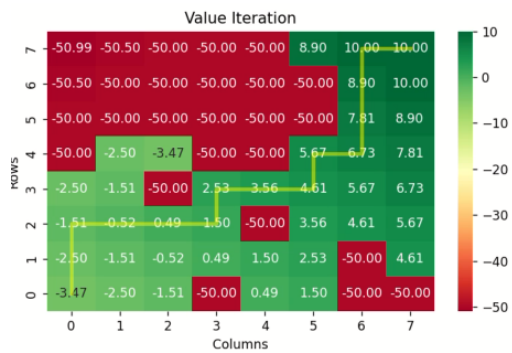

Figure 3: Value-function heatmap for **Value Iteration** (saved output).

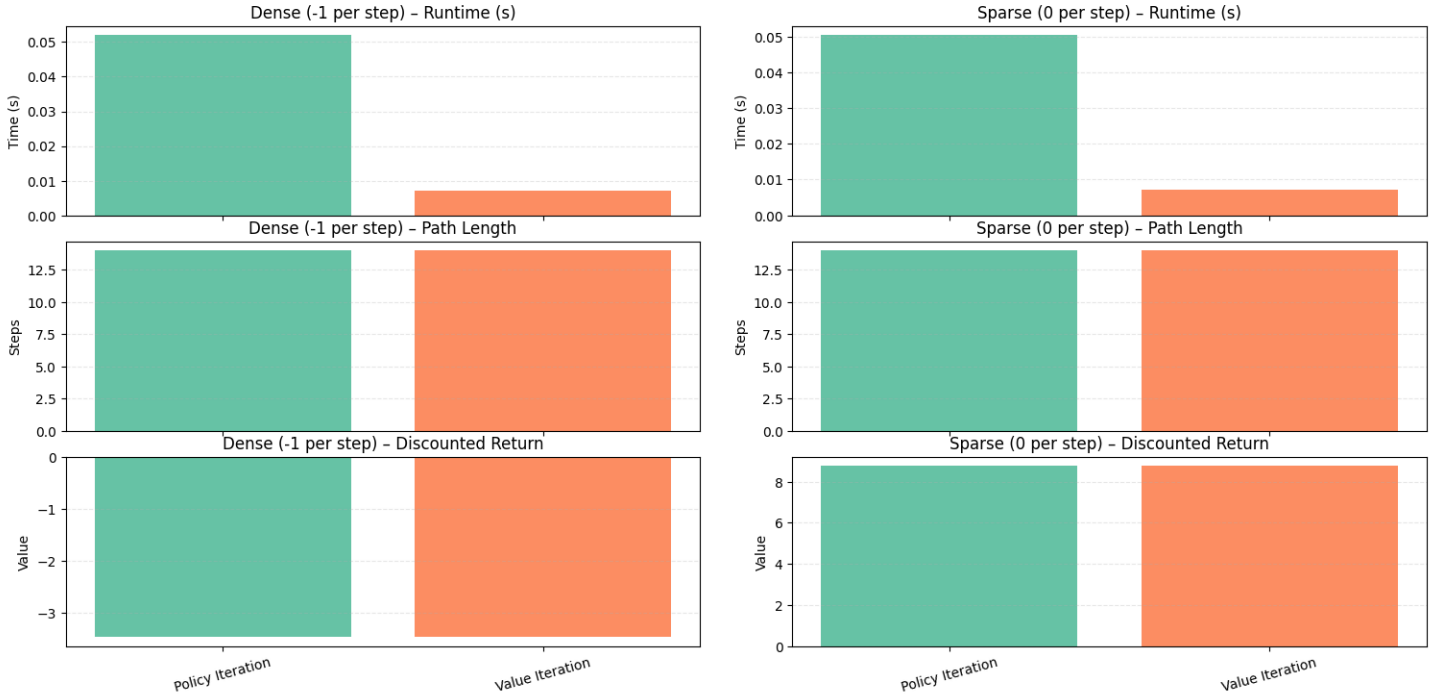

Figure 4: Comparison across reward scenarios and algorithms (runtime, path length, discounted return).

5.1 Discussion

- **Computation time:** In our runs, value iteration tends to have lighter-weight updates per sweep, while policy iteration spends more time in policy evaluation but can stabilize in fewer outer loops.
- **Path quality:** When both methods converge (same γ and stopping threshold), they typically produce the same optimal behavior (up to tie-breaking), so the final path reward is usually very close.
- **Effect of obstacles:** Because obstacle states terminate with a strong negative reward, the value function drops sharply near them, which naturally pushes the greedy policy to route around those cells.

6 Conclusion

Policy iteration and value iteration both successfully compute optimal policies for the grid navigation task. The DP solution integrates naturally with PyBullet visualization by converting state indices to 3D positions and commanding the UR5 end-effector through inverse kinematics.